

СПИСОК ИСПОЛНИТЕЛЕЙ

Руководитель НИР,
Инженер-исследователь

Н.А. Семенов

(Введение, раздел 1, 2, 3,

(подпись, дата)

4, 5 заключение)

Инженер-программист

М.А. Меньщиков

(раздел 1, 2, 3, 4, 5)

(подпись, дата)

Инженер-исследователь

Д.С. Евсеев

(раздел 1, 2, 3, 4, 5)

(подпись, дата)

Нормоконтроль

К.И. Макаренко

(подпись, дата)

РЕФЕРАТ

Отчёт 30 с., 1 кн., 2 рис., 2 табл., 15 источн.

ГРАФ, ПОИСК, ИЗВЛЕЧЕНИЕ, ПАМЯТЬ, ЛЛМ, ТРИПЛЕТ, ТЕЗИС, ПОИСКОВАЯ ДОПОЛНЕННАЯ ГЕНЕРАЦИЯ

Объект исследования НИР – построение и использование графов знаний с помощью ЛЛМ. Цель исследования – разработка эффективных алгоритмов построения и актуализации персонального графа знаний на основе слабоструктурированных источников информации. Основные задачи включают:

1. Обзор наиболее релевантных подходов (SOTA) построения, изменения и актуализации графов знаний с использованием ЛЛМ (раздел 2.2).
2. Разработка метода полуавтоматического построения графа знаний по бенчмарку DiaASQ (раздел 2.1).
3. Разработка алгоритма внесения в персональный граф знаний фактов и связей из структурированных источников (раздел 3).
4. Разработка алгоритма внесения в персональный граф знаний фактов и связей из слабоструктурированных источников с использованием ЛЛМ (раздел 3).
5. Разработка алгоритма актуализации графа (раздел 3).
6. Сравнение классических алгоритмов поиска на графе, в частности A со звездой и поиска в ширину (раздел 4).
7. Разработка нового, комбинированного алгоритма поиска на графе, специфического для выбранной структуры графа и включающего в себя как классические алгоритмы (A со звездой, поиск в ширину), так и поиск по эмбедингам (раздел 4).
8. Тестирование алгоритмов работы с графом на диалоговом бенчмарке DiaASQ (раздел 5).
9. Разработка библиотеки для программного кода алгоритмов (раздел 5).
10. Подготовка проекта публикации на английском языке, оформленной в стиле ведущей международной конференции с рейтингом A*, или в научном журнале с рейтингом Q1 (Приложение к настоящему отчету).

Данные решения позволят создать персонального ассистента, помнящего специализированную информацию о каждом пользователе.

Для разработки методов построения и использования графов знаний был произведен поиск существующих вариантов работы с графом при помощи ЛЛМ, графовых

алгоритмов поиска, бенчмарков работы с памятью, вариантов использования построенных вручную графов для улучшения качества работы ЛЛМ.

В рамках настоящей работы был предложен и протестирован алгоритм построения графа знаний, включающего триплеты, тезисы и эпизодические воспоминания. Также был предложен алгоритм обновления графа, то есть идентификации и удаления устаревших знаний. Наконец, были предложены и протестированы три алгоритма поиска информации в графе. Эти алгоритмы превосходили классическую поисковую дополненную генерацию и были испытаны на двух ЛЛМ: LLama-3-8B-Instruct [1] и GigaChat pro.

Результаты данных исследований предполагается применять для построения персонализированного помощника, собирающего информацию о пользователе (или группе пользователей) и способного использовать эти знания для ответов на вопросы. Такой помощник может генерировать саммари встреч, напоминать о задачах, искать информацию о проектах и т.д.

Основанием для проведения работ является Техническое задание по теме «Исследование алгоритмов создания и актуализации графов знаний на слабоструктурированных данных» (Приложение №4 к Дополнительному соглашению №5 от 13 августа 2024 г. к Договору с Индустриальным партнером №10835/3978620 от «26» августа 2021 г., далее – Техническое задание).

СОДЕРЖАНИЕ

СПИСОК ИСПОЛНИТЕЛЕЙ.....	2
РЕФЕРАТ	3
ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ	7
ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ	8
ВВЕДЕНИЕ	9
1 Алгоритмические формулировки задачи	11
1.1 Формулировка задачи построения графа знаний из слабоструктурированных источников	11
1.2 Формулировка задачи построения графа знаний из структурированных ис- точников	11
1.3 Формулировка задачи извлечения информации из графа	12
1.4 Формулировка задачи обновления графа	12
2 Формулировка гипотез о перспективных методах и подходах.....	13
2.1 Используемые источники данных	13
2.2 Модели и методы	14
3 Построение и актуализация графа знаний	16
4 Алгоритмы поиска информации в графе	17
4.1 Инициализация поиска	18
4.1.1 Разбор вопроса	18
4.1.2 Сопоставление вопроса со знаниями в графе.....	18
4.2 Алгоритм А со звездой	18
4.3 Алгоритм поиска в ширину	19
4.4 Генерация запроса к графу	21
4.5 Генерация ответа на вопрос	21
5 Вычислительные эксперименты	22
5.1 Поисковая дополненная генерация	22
5.1.1 Конфигурация поиска.....	22
5.1.2 Выводы.....	23
5.2 Эксперименты на истинном графе	23
5.2.1 Вопросно-ответная система на основе поиска в ширину и применения LLaMa и GigaChat для генерации текста	23
5.2.2 Вопросно-ответная система на основе Cypher-запросов и применения LLaMa и GigaChat для генерации текста	24
5.2.3 А со звездой	24
5.2.4 Выводы	24

5.3 Эксперименты на построенном графе	26
5.3.1 Вопросно-ответная система на основе поиска в ширину и применения LLaMa и GigaChat для генерации текста	26
5.3.2 А со звездой.....	26
5.3.3 Выводы	26
ЗАКЛЮЧЕНИЕ.....	28
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	30

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ

В настоящем отчёте о НИР применяют следующие термины и определения.

Агент — ИИ-инструмент (часто ЛЛМ), решающий конкретную поставленную задачу (так как постановок задач для ИИ много, агенты могут выполнять очень разные функции).

Граф знаний — структура памяти, хранящая информацию в упорядоченном (графовом) виде.

Cypher — язык запросов к графовым базам данных.

Триплет — элементарная структура графа знаний, состоящая из субъекта, отношения и объекта.

Тезис — элементарная структура графа знаний, состоящая из осмысленной фразы и списка объектов, фигурирующих в этой фразе.

Эпизодическое воспоминание — элементарная структура графа знаний, состоящая из отрывка текста и списка объектов, фигурирующих в этом отрывке.

DiaASQ — датасет, содержащий диалоги пользователей некторого форума и разметку этих диалогов.

A^* , A со звездой — алгоритм поиска на графе.

ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ

В настоящем отчёте о НИР применяют следующие сокращения и обозначения.

ЛЛМ — нейросетевая модель на основе архитектуры трансформер, способная генерировать текст, в частности - выполнять текстовые инструкции и отвечать на вопросы.

ИИ — искусственный интеллект.

SOTA — наилучшее решение поставленной задачи, доступное на данный момент.

BFS — алгоритм поиска на графе.

SQL — язык запросов к базам данных.

СУБД — системы управления базами данных.

QA — вопросы и ответы (question-answer).

Поисковая дополненная генерация — классический способ поиска релевантной информации для ответа на вопрос.

ВВЕДЕНИЕ

Программа Центра прикладного искусственного интеллекта Сколтеха предусматривает создание ПО, реализующего внешнюю память для ЛЛМ при помощи графов знаний. Внешняя память для ЛЛМ позволяет решить проблему восприятия персонального контекста, которая возникает при необходимости проанализировать большой объем текстов. Также данная технология может использоваться для создания персональных ИИ помощников, использующих данные о пользователе и о мире, не попавшие в обучающую выборку модели. Таким образом, внешняя память позволит использовать ЛЛМ более эффективно и персонализированно.

Для реализации такой памяти используются графы знаний [2, 3, 4, 5], которые строятся ЛЛМ в автоматическом режиме и поддерживают контекстуальные алгоритмы извлечения данных, намного превосходящие классическую поисковую дополненную генерацию по производительности. Более конкретно, граф имеет два основных функционала: запоминание произвольной текстовой информации и извлечение наиболее релевантных знаний по вопросу к нему. Такие инструменты удобны в использовании и могут быть применены к широкому спектру задач, связанных с ЛЛМ.

Графовое представление запоминаемой информации имеет несколько преимуществ. Во-первых, знания в графе представлены атомарно, что позволяет таргетно извлекать нужное и отсеивать нерелевантное. Во-вторых, такая структура весьма универсальна и способна выразить большую часть текстовой информации. В третьих, графовый формат делает возможным быстрый поиск, добавление и удаление знаний без глобальной перестройки всего накопленного объема. Эти операции в равной степени используются как в запоминании, так и во "вспоминании" и обновлении. Пример текста и графа, построенного из него, можно найти на рисунке 1.



Рисунок 1 — Пример графа

В рамках НИР решены следующие задачи:

1. Обзор наиболее релевантных подходов (SOTA) построения, изменения и актуализации графов знаний с использованием ЛЛМ (раздел 2.2).
2. Разработка метода полуавтоматического построения графа знаний по бенчмарку DiaASQ (раздел 2.1).
3. Разработка алгоритма внесения в персональный граф знаний фактов и связей из структурированных источников (раздел 3).
4. Разработка алгоритма внесения в персональный граф знаний фактов и связей из слабоструктурированных источников с использованием ЛЛМ (раздел 3).
5. Разработка алгоритма актуализации графа (раздел 3).
6. Сравнение классических алгоритмов поиска на графе, в частности А со звездой и поиска в ширину (раздел 4).
7. Разработка нового, комбинированного алгоритма поиска на графе, специфического для выбранной структуры графа и включающего в себя как классические алгоритмы (А со звездой, поиск в ширину), так и поиск по эмбедингам (раздел 4).
8. Тестирование алгоритмов работы с графом на диалоговом бенчмарке DiaASQ (раздел 5).
9. Разработка библиотеки для программного кода алгоритмов (раздел 5).
10. Подготовка проекта публикации на английском языке, оформленной в стиле ведущей международной конференции с рейтингом А*, или в научном журнале с рейтингом Q1.

1 Алгоритмические формулировки задачи

1.1 Формулировка задачи построения графа знаний из слабоструктурированных источников

В рамках данного проекта под слабоструктурированными источниками подразумевается любая текстовая информация. Для ее запоминания строится граф знаний, включающий максимально полное представление исходного текста и состоящий из следующих элементов:

1. Ноды, представляющие атомарные сущности, извлеченные из текста (зеленые сущности на рисунке 1);
2. Простые связи между сущностями, отражающие их взаимосвязь (зеленые связи на рисунке 1);
3. Тезисы, являющиеся полными атомарными мыслями, представленными в тексте, и играющие роль гиперсвязей между произвольным набором нод в графе (желтые сущности и связи на рисунке 1);
4. Эпизодические воспоминания, являющиеся исходным текстом, который запоминается в графе, и играющие роль гиперсвязей между произвольным набором нод в графе (синие сущности и связи на рисунке 1).

Во введенных выше терминах, задача построения графа из слабоструктурированных источников сводится к формулировке нод, простых связей между ними и тезисов с помощью ЛЛМ, парсингу ответа модели и сохранению полученной информации в некотором структурированном виде (например, в виде триплетов "объект - отношение - объект").

1.2 Формулировка задачи построения графа знаний из структурированных источников

Помимо запоминания текстов, хотелось бы иметь возможность запоминать также структурированную и верифицированную информацию (к примеру, факт трудоустройства работника, посещение онлайн-встречи, заключение некоторого договора и тп). Такие данные имеют свои особенности, в частности, они редко выражены в виде исчерпывающего текста и намного чаще - в виде некоторых цифровых структур данных. Тем не менее, добавление подобной информации в граф расширило бы его верифицированными знаниями, что положительно сказалось бы на качестве извлекаемых из него знаний. Перевод цифровых структур данных в единый графовый формат (например, в

формат триплетов "объект - отношение - объект") и последующее добавление их в граф является содержанием задачи построения графа из структурированных источников.

1.3 Формулировка задачи извлечения информации из графа

Основной сценарий использования построенного графа знаний - извлечение из него информации, релевантной для ответа на запрос. В силу того, что прямой подсчет релевантности того или иного знания не представляется возможным, в данном сценарии неизбежно будет возникать баланс между релевантностью и полнотой извлекаемой информации (все, что извлекли, релевантно, но извлекли не все, что нужно, либо извлекли все, что нужно, но при этом еще какой-то шум). Таким образом, задача извлечения информации из графа состоит в нахождении алгоритмов, позволяющих регулировать баланс между полнотой и релевантностью и осуществлять извлечение знаний согласно выбранным параметрам.

1.4 Формулировка задачи обновления графа

В некоторых случаях сохраненные в графе знания теряют свою актуальность. Например, если яблоко лежало на столе и соответствующая информация была помещена в граф, то после того, как человек возьмет это яблоко в руки, в граф нужно будет записать информацию о том, что яблоко у человека, а старые данные забыть. Данный подход служит альтернативой простому запоминанию времени, в которое знание было помещено в граф, и является постановкой задачи обновления графа.

2 Формулировка гипотез о перспективных методах и подходах

В этом разделе перечислены используемые данные и методы, а так же кратко дано обоснование их применения. Более подробное описание методов и экспериментальные результаты их работы будут описаны в соответствующих разделах.

2.1 Используемые источники данных

Для тестирования алгоритмов запоминания информации используется диалоговый бенчмарк DiaASQ [6]. Он состоит из диалогов пользователей китайского форума о характеристиках различных устройств, а также снабжен структурированными высказываниями, содержащими смысл диалогов. Данные высказывания используются в качестве истинного графа, так как легко переводятся в формат триплетов. На основании этих истинных высказываний процедурно был сгенерирован пул вопросов к данным диалогам, которые используются для тестирования всех алгоритмов данной работы. По своей структуре вопросы разделены на 10 типов.

compare questions

Сравнение двух устройств по заданному параметру.

Пример: Which device is better in battery life: Apple or k30u?

device sentiment

Вопрос о мнении пользователя о заданном устройстве в заданный момент времени.

Пример: Kayla has positive, negative or neutral opinion about video of 10PRO on 25.11.2020?

compare sentiment

Модель должна определить по извлеченным триплетам, какое мнение пользователей об устройстве преобладающее.

Пример: The majority of speakers have positive, neutral or negative sentiment about system of vivo?

same devices

Нужно найти общее устройство двух пользователей.

Пример: Do Brianna and Joseph have any common devices (which Brianna and Joseph both use)? If so, list common devices. Otherwise, answer 'No'.

same manufacturers

Нужно найти устройства одного и того же производителя двух пользователей. Данный вопрос требует поиск глубины 2 в графе.

Пример: Do Lily and Jenna prefer the same device manufacturer?

similar device opinions

Нужно определить, мнения какого из двух пользователей более похожи на мнение заданного пользователя об устройствах.

Пример: Whose opinions from Anthony and Grace about devices are most similar to Faith's?

similar manuf opinions

Вопрос аналогичен similar device opinions, только о производителях устройств.

Пример: Whose opinions from Evan and James about manufacturers are most similar to Abigail's?

which people about devices

Вопрос о пользователе, который имеет заданное мнение об устройстве в заданную дату.

Пример: Which people have negative opinion about video of 10PRO on 25.11.2020?

dominant opinion

Вопрос о преобладающем мнении пользователя о компоненте устройства.

Пример: What Jessica's opinion (positive, negative or neutral) about signal of Apple was dominant during using Apple?

last opinion

Вопрос о последнем мнении пользователя о компоненте устройства.

Пример: What opinion (positive, negative or neutral) about scheduling of IQOO9 was last during Zachary's experience of IQOO9?

2.2 Модели и методы

Языковые модели Во многих этапах построения и использования графа применяются запросы к ЛЛМ, а также подсчет эмбеддингов. Данные операции не имеют альтернатив в предлагаемом подходе, поэтому доступ к качественной ЛЛМ и модели-эмбеддеру является необходимым. В качестве ЛЛМ в настоящем исследовании фигурировали GPT-4o mini, GigaChat pro, LLaMA-3-8B-Instruct. В качестве моделей-эмбеддеров - mContriever и e5-small.

Поисковая дополненная генерация В качестве бейзлайна, с которым сравнивается графовый подход, выступает поисковая дополненная генерация, извлекающая диалоги и их фрагменты из (reference) для ответов на сгенерированные вопросы.

Использование графов знаний Конечно, графы знаний уже используются [7] для повышения качества работы ЛЛМ. Данные работы можно разделить на те, которые дообучаются на использование конкретного типа графов [8, 9], работы, в которых модели не дообучаются, но используют граф, построенный вручную [10, 11, 12, 13], а также исследования, в которых граф строится при помощи ЛЛМ без дообучения последней [2, 3, 4, 5].

Алгоритм извлечения знаний из слабоструктурированных источников Для генерации структурированного представления знаний из текста используется запрос к ЛЛМ, промпт-инжиниринг для повышения качества результата и парсинг извлеченной информации. [14].

Алгоритм поиска в ширину Алгоритм поиска в ширину (BFS) используется для обхода или поиска вершин в графе или дереве. Он начинается с корня или начальной вершины и исследует соседние вершины на текущем уровне перед переходом к вершинам на следующем уровне. BFS, как правило, реализуется с использованием очереди, которая помогает отслеживать порядок посещения вершин. Этот алгоритм особенно полезен для нахождения кратчайшего пути в графах без взвешенных рёбер или для обхода всех вершин одного уровня перед движением глубже.

Алгоритм A со звездой A^* — это алгоритм поиска пути, который используется для нахождения кратчайшего пути от начальной вершины до конечной в графах с неотрицательными весами ребер. Этот алгоритм основан на методе динамического программирования и является улучшенным вариантом алгоритма Дейкстры. Основная идея A^* заключается в том, что он использует функцию оценки стоимости достижения цели, которая учитывает как уже пройденный путь, так и оставшийся путь до целевой вершины. Эта функция позволяет алгоритму выбирать наиболее перспективные вершины для дальнейшего исследования, сокращая количество шагов для нахождения оптимального пути.

Генерация запроса к графу В отличие от алгоритмов поиска в ширину и A со звездой, выполнение Cypher-запроса к графу позволяет получить только релевантные триплеты (тогда как при поиске в ширину и A со звездой часть триплетов могут не относиться к вопросу). Для генерации Cypher-запроса составляем промпт для ЛЛМ, который включает в себя инструкцию сгенерировать запрос и несколько in-context примеров вопросов и соответствующих Cypher-запросов.

Алгоритм обновления графа Для обнаружения устаревших знаний на каждом шаге запоминания запускается поиск в ширину, начинающийся с тех сущностей, которые были найдены в запоминаемом тексте. Далее, знания, извлеченные поиском в ширину вместе с только что запомненными помещаются в специальный промпт, который просит ЛЛМ обнаружить потенциальные замены старых знаний на новые. Та информация, которая может быть заменена, считается устаревшей и удаляется. Такой подход позволяет обновлять только ту часть графа, которая претерпевает изменения при конкретном запоминании, не трогая все остальные знания.

3 Построение и актуализация графа знаний

Первым этапом запоминания информации является извлечение из текста триплетов с помощью ЛЛМ и специального промпта. Триплет состоит из субъекта, отношения и объекта. Например, из текста "Корабельщики в ответ: «Мы объехали весь свет; За морем житье не худо; В свете ж вот какое чудо" будут извлечены триплеты [корабельщики, объехали весь, свет], [житье, не худо за, море], [чудо, есть в, свет]. Далее субъекты и объекты триплетов становятся нодами графа знаний, а отношение между ними - простой связью в нем.

Далее по аналогичному сценарию происходит извлечение тезисов и их сущностей. Тезис, как и триплет, содержит некоторую атомарную информацию из текста, однако, в отличие от триплета, может объединять произвольное количество сущностей и всегда содержит законченную мысль, не зависящую от контекста. Например, из текста "За морем житье не худо; В свете ж вот какое чудо: Остров на море лежит, Град на острове стоит" будут извлечены тезисы "За морем житье не худо" "В свете чудо - остров на море лежит" "В свете чудо - град на острове стоит" со списками сущностей [море, житье], [свет, чудо, остров, море], [свет, чудо, град, остров] соответственно. Такая гиперсвязь также помещается в граф.

Наконец, весь исходный текст также помещается в граф как гиперсвязь между всеми сущностями, фигурирующими в извлеченных триплетах и тезисах. Такая гиперсвязь, по аналогии с когнитивными науками, называется эпизодической.

Поиск устаревшей информации в графе производится следующим образом. В начале множество сущностей, фигурирующих в только что извлеченных триплетах и тезисах сопоставляется с существующими вершинами графа для обнаружения совпадений. Если таковые присутствуют, то они являются стартовым набором нод для поиска в ширину, который извлекает все связи и гиперсвязи, начинающиеся или заканчивающиеся в этих нодах. Далее, с помощью специального промпта ЛЛМ пытаются заменить найденные знания только что извлеченными. Если какие-то триплеты или тезисы смогли быть заменены, то они удаляются из графа.

Для добавления структурированной информации реализован алгоритм, ожидающий список знаний в определенном формате, и добавляющий их в существующий граф знаний.

4 Алгоритмы поиска информации в графе

Общий конвейер по извлечению релевантной информации из графа знаний для генерации ответов на вопросы представлен на рисунке 4.1.

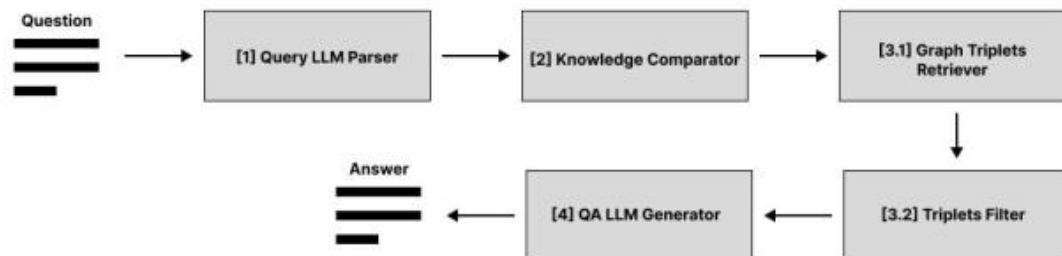


Рисунок 4.1 — Конвейер поиска информации в графе для генерации ответов на вопросы

Из Рисунка 4.1 видно, что конвейер разбит на четыре стадии, при этом третья стадия состоит из двух этапов. На вход конвейеру поступает пользовательский вопрос в строковом формате, который передаётся в модуль "Query LLM Parser". В данном модуле выполняется извлечение ключевых сущностей из вопроса. На второй стадии полученная информация передаётся в модуль "Knowledge Comparator", где выполняется сопоставление (matching) сущностей из вопроса с объектами из графа знаний. На третьей стадии выполняется поиск и фильтрация триплетов из графа знаний, релевантных вопросу, в рамках первого и второго этапов соответственно. На первом этапе с помощью модуля "Graph Triplets Retriever" выполняется запуск одного из реализованных алгоритмов обхода графа: в качестве стартовых выступают те вершины, которые были сопоставлены сущностям из вопроса в рамках второй стадии конвейера. В результате будет извлечён список потенциально-релевантных триплетов. На втором этапе с помощью модуля "Triplets Filter" выполняется ранжирование триплетов по степени их семантической близости к вопросу. В результате работы третьей стадии будет получен список из N (гиперпараметр) триплетов, ближайших (семантически) к данному вопросу. На последней четвёртой стадии с помощью модуля "QA LLM Generator" выполняется генерация ответа на вопрос, обусловленная извлечённой информацией. Полученный ответ в строковом формате возвращается как результат работы конвейера.

Данная архитектура конвейера мотивирована следующими соображениями: Для получения хорошего начального приближения к подграфу с требуемой информацией необходимо сопоставить ключевые объекты из вопроса с похожими объектами из имеющегося набора знаний. Информация, которая требуется для генерации правильного ответа содержится в том же подграфе, что и ключевые сущности из вопроса. Извлечённые из графа знаний триплеты слабо обусловлены исходным вопросом. При этом

каждый триплет имеет свою степень важной информации, которая требуется для генерации правильного ответа. Также у ЛЛМ есть ограничение по максимальной длине текстовой последовательности, которую можно подать на вход.

Далее в подразделах представлено более подробное описание модулей, используемых в рамках данного конвейера.

4.1 Инициализация поиска

4.1.1 Разбор вопроса

На данной стадии выполняется обращение к ЛЛМ для извлечения сущностей из вопроса в авторегрессионном режиме. Модель на вход получает сам текстовый вопрос и prompt-подсказку, в котором описана задача и формат генерации результата.

4.1.2 Сопоставление вопроса со знаниями в графе

Для сопоставления вершин графа знаний с сущностями вопроса выполняется их перевод из текста в пространство эмбедингов: к полученным векторам дополнительно применяется оператор нормировки. Для каждой сущности с помощью алгоритма приближенного поиска извлекается M (гиперпараметр) самых ближайших вершин по “1 - dot-product”-метрике [15]. Полученные вершины сохраняются в единый список без информации об отношении к сущностям. В качестве embedder-модели используется E5-small из HuggingFace-репозитория.

Далее описаны три алгоритма, реализующих поиск по графу.

4.2 Алгоритм A со звездой

В качестве одного из алгоритмов обхода графа знаний для извлечения релевантных триплетов был реализован A^* -алгоритм. В рамках алгоритма выполняется поиск кратчайших путей между всеми парами вершин, сопоставленных сущностям на второй стадии QA-конвейера. В результате такого обхода все триплеты, попавшие в кратчайшие пути сохраняются в единый список с удалением дубликатов (по их содержанию). При использовании данного алгоритма выдвигается следующая гипотеза: информация, которая требуется для генерации правильного ответа содержится в триплетах из кратчайших путей между сущностями исходного вопроса. Во время обхода граф считается невзвешенным и неориентированным. В качестве d-метрики расстояния между смежными вершинами используется константное значение. В качестве эвристической h-метрики расстояния между текущей и конечной вершиной было реализовано три варианта:

1. “ip” - dot-product расстояние эмбедингов между текущей и конечной вершинами.

2. "weight with short path" - длина кратчайшего пути, полученного с помощью bfs-алгоритма, умноженная на "ip"-метрику.
3. "avg weighted with short path" - усреднённое значение ip-расстояний эмбедингов смежных вершин в пути от начальной до текущей и ip-расстояния между текущей и конечной вершинами, умноженное на длину кратчайшего пути, полученного с помощью bfs-алгоритма, между текущей и конечной вершинами.

В качестве модели для получения векторных представлений текста используется E5-small из HuggingFace-репозитория.

4.3 Алгоритм поиска в ширину

Один из методов для извлечения триплетов для ответа на вопрос, применяемых в данной работе, - это алгоритм поиска в ширину. Сначала происходит определение узлов в графе, соответствующих сущностям, извлеченным из вопроса. На первой итерации рассматриваются вершины, соседние с вершинами сущностей из вопроса, на последующих итерациях, вершины, соседние с найденными на предыдущем шаге. Таким образом, от каждой из вершин сущностей из вопроса происходит построение путей в графе во все стороны, и при пересечении двух путей от разных сущностей триплеты из данных путей записываются в итоговый список. В другой список записываются все извлеченные триплеты при движении по графу от сущностей из вопроса. В итоге алгоритм возвращает N триплетов из первого списка и K триплетов из второго, где N и K - настраиваемые гиперпараметры.

В случае построенного графа, который содержит не только триплеты, соединяющие сущности (типа "simple"), но и связи между сущностями и фрагментами текста (связи типа "hyper" и "episodic") используется следующая модификация алгоритма поиска в ширину:

1. построение путей в графе начинается с сущностей из вопроса, аналогично случаю с триплетами типа "simple" которые связывают сущности типа "simple";
2. в фрагментах текста, полученных при движении по графу от сущностей из вопроса, производится поиск подстрок с сущностями из вопроса, не совпадающими с сущностью, с которой начинался путь в графе до данного фрагмента текста. Для каждого фрагмента текста подсчитывается $N_{intersections}$ - число найденных сущностей в тексте.
3. Список триплетов типа "hyper" и "episodic" сортируется в порядке убывания $N_{intersections}$.

Для извлечения сущностей, соседних с данной сущностью, на каждом шаге поиска в ширину, в случае использования БД Neo4j выполняется один из двух Cypher-запросов:

1. 'MATCH (a:object)-[r]-(b:object) WHERE a.name="{name1}" RETURN a, r, b', где "object" тип сущности ("simple", "hyper" или "episodic"), данный запрос извлекает объект в триplete (субъект, отношение, объект) при известном субъекте.
2. 'MATCH (a:object)-[r]-(b:object) WHERE b.name="{name2}" RETURN a, r, b', данный запрос извлекает субъект при известном объекте.

В случае, если для данной сущности из вопроса компонент KnowledgeComparator извлекает несколько сущностей в графе (к примеру, для вопроса "Which device is better in battery: Xiaomi Mi12 or Iphone13?" могут быть извлечены сущности "battery", "accumulator", "power", и т. д., так как эмбединги слов "accumulator" и "power" близки в пространстве эмбедингов к вектору слова "battery"), эти сущности рассматриваются совместно как одна группа сущностей. Это значит, что алгоритм не учитывает пересечения путей в графе, которые начинаются с сущностей из одной группы (к примеру, "battery" и "accumulator").

В компоненте BFSRetriever для извлечения триплетов с помощью алгоритма поиска в ширину предусмотрено два режима:

1. `strict_filter = true` - в этом случае функция `get_relevant_triplets` возвращает только триплеты с отношениями типа "simple" из путей в графе, которые начинаются от разных сущностей и пересекаются между собой, и триплеты с отношениями типов "hyper" и "episodic", если в фрагментах текста из этих триплетов было найдено более одной сущности из вопроса. Данный режим позволяет минимизировать количество нерелевантных триплетов.
2. `strict_filter = false` - в этом случае не происходит фильтрации триплетов, как в предыдущем случае. Для построенных графов, для которых на большинстве вопросов извлекается 0 триплетов, рекомендуется использовать данный режим извлечения.

Кроме того, в конфигурационном классе `BFSSearchConfig` предусмотрены следующие гиперпараметры, чтобы регулировать количество и состав извлекаемых триплетов:

1. `hyper_episodic_num` - число возвращаемых триплетов типа "hyper" и "episodic".
2. `chain_triplets_num` - число триплетов типа "simple" из пересекающихся путей в графе, начинающихся от разных сущностей из вопроса.
3. `other_triplets_num` - число триплетов типа "simple", не входящих в пересекающиеся пути в графе.

4.4 Генерация запроса к графу

Cypher представляет собой SQL-подобный открытый язык запросов к графовым СУБД. Основными элементами, которыми оперирует этот язык, являются вершины (ноды) и рёбра графа. Рёбра в Neo4j имеют тип и направление, вершины же могут быть помечены одной или более метками, а так же могут иметь несколько дополнительных свойств.

Для генерации Cypher-запроса к графу используется промпт, содержащий инструкцию и in-context пример вопроса и соответствующего Cypher-запроса. Для каждого из вопросов происходит поиск наиболее подходящего по структуре вопроса. Каждый вопрос из in-context примеров векторизуется с помощью предобученного трансформера (Contriever). Для вопроса на входе модели выполняется вычисление скалярного произведения эмбединга вопроса с каждым из эмбедингов вопросов из in-context примеров. В промпт добавляется in-context пример с наибольшим скалярным произведением.

Для вопросов типов `compare_questions`, `similar_device_opinions`, `similar_manf_opinions` генерируются несколько Cypher-запросов, так как для `compare_questions` необходимо извлечение триплетов о нескольких устройствах, а для `similar_device_opinions` и `similar_manf_opinions` - триплетов о предпочтениях нескольких людей.

4.5 Генерация ответа на вопрос

На данной стадии выполняется обращение к ЛЛМ для генерации ответа на вопрос в авторегрессионном режиме. Модель на вход получает сам текстовый вопрос, список извлечённых триплетов с третьей стадии, приведённых в строковый формат, и prompt-подсказку, в котором описана задача и формат генерации результата.

5 Вычислительные эксперименты

Все методы, используемые на датасете DiaASQ, оценивались по одной и той же схеме: для каждого вопроса из сгенерированного пула производился поиск релевантной информации (в диалогах для поисковой дополненной генерации, в истинном графе для экспериментов на нем, в построенном графе для экспериментов на нем), после эта информация поступала в промпт модели, которая генерировала ответ. Данный ответ сравнивался с правильным (верно / неверно, при помощи посимвольного сравнения), и в конце вычислялось значение Ассигасы по всем вопросам. В итоговых таблицах 5.1, 5.2 приведены основные экспериментальные результаты (L - LLama-3-8B-Instruct, G - GigaChat pro).

5.1 Поисковая дополненная генерация

5.1.1 Конфигурация поиска

Для использования классической поисковой дополненной генерации имеющийся массив текстовой информации необходимо разделить на чанки, которые и будут извлекаться в процессе поиска. В используемом датасете это можно сделать минимум двумя естественными образами: за один чанк можно взять диалог или реплику диалога. Соответственно, стратегий извлечения можно предложить уже три: извлекать только диалоги, только реплики или какое-то количество диалогов вместе с каким то количеством реплик.

Также для оптимальной работы важно количество чанков, извлекаемых с помощью модели-эмбеддера. В наших экспериментах при извлечении чанков использовался эмбеддер mContriever, для каждой из трех стратегий мы тестировали пять различных количеств извлечения: для диалогов - 3, 5, 7, 10 и 15, для реплик - 5, 10, 15, 20 и 25, для смешанного режима (количество реплик + количество диалогов) - $5 + 2$, $7 + 3$, $10 + 5$, $12 + 7$ и $15 + 10$. Итого получилось 15 различных сеттингов извлечения. Далее, для каждого из 10 типов вопросов мы выбирали лучший (с точки зрения значения метрики) сеттинг и использовали результат на нем как качество работы поисковой дополненной генерации на данном типе вопросов. Таким образом, значения поисковой дополненной генерации, представленные в таблице, в каком-то смысле оптимальны и не могут быть ощутимо улучшены подбором гиперпараметров.

Наконец, мы сравнивали две ЛЛМ, которые с помощью извлекаемой эмбеддером информации давали ответы на вопросы: GigaChat pro и LLaMA-3-8B-Instruct. Данное сравнение важно для интерпретации результатов, полученных в следующих экспериментах. Значения метрик каждой модели на каждом типе вопросов приведены в первых

двух столбцах (RAG L и RAG G, соответственно) таблицы 5.1.

5.1.2 Выводы

Полученные результаты важны главным образом для сравнения с графовыми методами, это сравнение приведено в следующих разделах отчета. Единственный вывод, который можно сформулировать по данному эксперименту - LLaMA по сравнению с Gigachat ощутимо лучше работает со средним и длинным контекстом, то есть меньше теряется в объеме информации и меньше галлюцинирует. В силу этого ее результаты почти на всех типах вопросов превосходят результаты Gigachat.

5.2 Эксперименты на истинном графе

5.2.1 Вопросно-ответная система на основе поиска в ширину и применения LLaMa и GigaChat для генерации текста

Истинный граф содержит триплеты для диалогов из датасета DiaASQ. Сущности в триплетах могут относиться к одному из трех лейблов:

1. person (имя человека, к примеру, Alfred);
2. device (названия устройства, к примеру, Xiaomi 10Pro);
3. feature (свойство устройства, к примеру, battery life, screen, video).

Отношения между сущностями принадлежат к одному из трех лейблов:

1. has_device (Пример: Alfred, has_device, Xiaomi 10pro);
2. opinion (Пример: Xiaomi 10Pro, opinion, battery life, свойство opinion: good);
3. manufacturer (Пример: Iphone 12, manufacturer, Apple).

Отношения типа opinion имеют следующие свойства:

1. time (время диалога);
2. person (имя человека, который сделал данное высказывание об устройстве);
3. opinion (мнение об устройстве).

Основа данного графа предоставляется датасетом, однако некоторые данные (например, имя спикеров или время диалога) были сгенерированы в ходе настоящей работы дополнительно для проверки некоторых свойств алгоритмов поиска на графе.

Результаты поиска в ширину по истинному графу представлены в таблице 5.1 (столбцы BFS G и BFS L).

5.2.2 Вопросно-ответная система на основе Cypher-запросов и применения LLaMa и GigaChat для генерации текста

В данной конфигурации экспериментов ответ на вопросы состоял из следующих шагов:

1. Генерация по вопросу Cypher-запроса с помощью ЛЛМ.
2. Выполнение Cypher-запроса и извлечение триплетов.
3. Генерация ответа по вопросу, извлеченным триплетам и in-context примерам.

Так как в Cypher-запрос должны входить сущности, совпадающие с сущностями из графа (иначе по запросу будет извлечен пустой список триплетов), на вопросах типов "compare sentiment synonyms" и "which people about device synonyms" не было проведено экспериментов с генерацией Cypher-запросов. В вопросах "compare sentiment synonyms" и "which people about device synonyms" названия свойств устройств были заменены на синонимы с помощью ЛЛМ (к примеру, "battery" было заменено на "accumulator"). В случае с использованием поиска в ширину для сущностей, извлеченных из вопроса, происходил поиск с помощью векторной базы данных сущностей в графе, близких по значению скалярного произведения с вектором сущности из вопроса. Данный подход позволяет находить триплеты для сущностей, выраженных в вопросе синонимами.

Результаты поиска с использованием Cypher-запроса представлены в таблице 5.1 (столбцы Cypher G и Cypher L).

5.2.3 A со звездой

В данном эксперименте выполняется оценка качества QA-конвейера на истинном графе при использовании A*-алгоритма в рамках третьей стадии. В качестве ЛЛМ тестировались два варианта: Llama и Gigachat. В случае использования Llama-модели для A*-алгоритма были заданы следующие значения гиперпараметров: d-метрика – "ip"; h-метрика – "ip". В случае Gigachat-модели гиперпараметры A*-алгоритма имели следующие значения: d-метрика – "constant"; h-метрика – "ip".

Результаты поиска с использованием A со звездой представлены в таблице 5.1 (столбцы A* G и A* L).

5.2.4 Выводы

При тестировании на истинном графе вопросно-ответная система на основе генерации Cypher-запросов показывает более высокие метрики, так как на графе, имеющем небольшое количество возможных типов сущностей и отношений, выполнение Cypher-запроса приводит к извлечению только релевантных триплетов (тогда как при поиске

в ширину в итоговый список триплетов могут входить нерелевантные триплеты, что осложняет генерацию ответа).

На вопросах типа "compare questions" модели показывают метрики выше, чем на других типах вопросов, так как в данном типе вопроса требуется выбрать, какое из двух устройств лучше по данному параметру (Пример: "Which device is better in battery life: iPhone11 Pro Max or Xiaomi 12U?"). В обоих случаях (BFS и Cypher-запросы) удается точно извлечь триплеты, содержащие название устройства и свойство устройства, и соответственно, моделям далее не сложно выбрать правильный ответ из двух вариантов.

На вопросах типов "same devices" и "similar device opinions" модели также показывают более высокие метрики, чем для других типов вопросов. В данном случае требуется найти общие устройства двух людей. При поиске в ширину мы находим пересечение двух путей в графе от сущностей, обозначающих двух людей. Cypher-запрос для данного типа вопроса содержит имена двух людей, и такой запрос позволяет однозначно извлечь триплеты с ответом.

В целом можно отметить, что графовые методы (за исключением A со звездой, предназначенного для поиска небольшого числа длинных взаимосвязей и использующегося на данной задаче как референс) справились с задачей ощутимо лучше поисковой дополненной генерации даже при условии, что для последней подбирались оптимальные гиперпараметры под каждый тип вопросов.

Таблица 5.1 — Таблица с метриками на построенном графе. Нумерация вопросов совпадает с порядком, представленным в используемых источниках данных. L - результаты Llama, G - результаты GigaChat

	RAG L	RAG G	BFS L	BFS G	Cypher L	Cypher G	A* L	A* G
1	0.68	0.66	0.94	0.88	0.82	0.48	0.63	0.56
2	0.5	0.48	0.54	0.50	0.42	0.48	0.55	0.46
3	0.6	0.38	0.66	0.60	0.62	0.44	0.47	0.25
4	0.48	0.52	0.85	0.81	0.90	0.86	0.50	0.28
5	0.74	0.58	0.81	0.74	0.97	0.55	0.03	0.38
6	0.38	0.37	0.64	0.79	0.72	0.62	0.61	0.15
7	0.59	0.41	0.60	0.26	-	-	0.37	0.21
8	0.34	0.28	0.54	0.82	-	-	0.32	0.08
9	0.59	0.59	0.68	0.62	0.64	0.54	0.48	0.37
10	0.68	0.65	0.74	0.74	0.72	0.68	0.66	0.46

5.3 Эксперименты на построенном графе

5.3.1 Вопросно-ответная система на основе поиска в ширину и применения LLaMa и GigaChat для генерации текста

Построенный граф формируется путем извлечения триплетов и эпизодов из диалогов и последующего сохранения извлеченных паттернов в графовую базу данных. Построенный граф отличается от истинного следующими элементами:

1. при построении графа не регулируются лейблы сущностей и отношений (у истинного графа у сущностей один из трех лейблов ("person", "device", "feature") и 3 лейбла отношений ("has device", "opinion", "manufacturer"));
2. истинный граф содержит один тип отношений ("simple"), а построенный - три типа ("simple", "hyper", "episodic").

Для построенного графа, в отличие от истинного, меньше вероятность найти ответ в триплетах с отношениями типа "simple", так как триплеты построенного не имеют структурированной онтологии. С другой стороны, в триплетах типа "episodic" и "hyper" содержатся фрагменты текста, которые более удобны для обработки ЛЛМ при генерации ответа.

Результаты работы вопросно-ответной системы с использованием построенного графа и алгоритма поиска в ширину представлены в таблице 5.2 (столбец BFS G).

5.3.2 A со звездой

В данном эксперименте выполняется оценка качества QA-конвейера на построенном графе при использовании A*-алгоритма в рамках третьей стадии. Для A*-алгоритма были заданы следующие значения гиперпараметров: d-метрика – "constant"; h-метрика – "ip"; максимальная глубина обхода графа для поиска конечной вершины – 10; максимальное количество вершин графа, которые можно обойти для поиска конечной вершины – 500.

Результаты работы вопросно-ответной системы с использованием построенного графа и алгоритма поиска A со звездой представлены в таблице 5.2 (столбец A* G).

5.3.3 Выводы

Модели на истинном графе показывают более высокие метрики, чем на построенном, так как истинный граф имеет четкую структуру онтологии (сущности определенных лейблов соединены между собой отношениями определенных лейблов), тогда

как построенный граф значительно менее детерминированный. Соответственно, вероятность найти триплеты с ответом на вопрос в построенном графе меньше, чем в истинном. Также набор извлеченных триплетов из построенного графа содержит больше нерелевантных триплетов (не содержащих ответ на вопрос), чем набор триплетов из истинного графа. Преимущество построенного графа состоит в универсальности: не нужно заранее задавать лейблы сущностей и отношений. Повышение метрик на построенном графе возможно, если зафиксировать структуру графа при построении путем добавления в промпт для извлечения триплетов соответствующих in-context примеров.

Тем не менее, результаты на построенном графе превосходят поисковую дополненную генерацию с оптимальными гиперпараметрами.

LLaMa-3-8B-instruct показывает более высокие метрики, чем GigaChat pro, так как LLaMa лучше понимает инструкции в промпте. К примеру, при добавлении в промпт инструкции "None answers are strictly prohibited" LLaMa в большинстве случаев генерирует ответы, отличные от "None", тогда как GigaChat игнорирует инструкцию. Также достаточно частый ответ GigaChat - "unable to determine" или "difficult to determine", который в случае LLaMa не встречается. Наконец, датасет сформулирован на английском языке, тогда как GigaChat заточен на работу с русским языком.

Таблица 5.2 — Таблица с метриками на построенном графе. Нумерация вопросов совпадает с порядком, представленным в используемых источниках данных. L - результаты LLaMa, G - результаты GigaChat

	RAG G	BFS G	A* G
1	0.66	0.78	0.25
2	0.48	0.70	0.25
3	0.38	0.47	0.24
4	0.52	0.48	0.48
5	0.58	0.47	0.38
6	0.37	0.37	0.02
7	0.41	0.27	0.17
8	0.28	0.33	0.01
9	0.59	0.59	0.37
10	0.65	0.60	0.40

ЗАКЛЮЧЕНИЕ

В ходе проведенной НИР был разработан алгоритм построения и обновления графа знаний при помощи ЛЛМ, реализованы классические методы поиска на графе, предложен новый алгоритм поиска в графе знаний, алгоритм внесения структурированных данных в граф знаний, подготовлен проект научной публикации, содержащей основные теоретические результаты проведенного исследования.

Предложенный алгоритм построения и обновления графа подразумевает запросы к ЛЛМ, парсинг результатов этих запросов и занесение информации, структурированной таким образом, в граф (или удаление устаревшей по мнению ЛЛМ информации из графа). Два предложенных алгоритма поиска не используют никаких предположений о структуре графа и могут быть выполнены с произвольным вопросом к произвольному графу, третий содержит некорректные предположения и может быть выполнен с произвольным вопросом к графу, удовлетворяющему этим предположениям.

Вычислительные эксперименты, проведенные на истинном графе бенчмарка DiaASQ, демонстрируют серьезное превосходство графовых методов над классической поисковой дополненной генерацией. Данный результат стабилен вне зависимости от типов обрабатываемых вопросов и используемой ЛЛМ.

Также, помимо истинного графа, были проведены эксперименты на графе, построенном с помощью ЛЛМ. Для этого эксперимента в качестве модели, строящей граф, был использован GigaChat pro. На построенном графе тестировались два алгоритма поиска, не требующих выполнения предположений о графе, так как построенный с помощью ЛЛМ граф не может гарантировать выполнение этих предположений. Результаты также говорят о превосходстве построенного графа над поисковой дополненной генерацией вне зависимости от использованной ЛЛМ, хотя также свидетельствуют и об отставании построенного графа от истинного.

Научно-технический уровень выполненной НИР соответствует актуальному уровню (SOTA) в данной области, в работе исследовались наиболее современные ЛЛМ, методы построения графа и методы поиска в графе.

Задачи, сформулированные в ТЗ к данному этапу НИР решены полностью. Рекомендуется тестирование разработанных алгоритмов на данных клиентов Индустриального Партнера.

Значимость данной работы заключается в разработке нового подхода для реализации памяти ЛЛМ. Такая память востребована во многих приложениях уже сейчас и активно реализуется с помощью поисковой дополненной генерации, однако предложенный метод существенно улучшает качество внешней памяти. С его помощью могут быть реализованы персональные ассистенты, качественные суммаризаторы, умные хранилища слабоструктурированной информации и прочее.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Dubey A., Jauhri A., Pandey A., Kadian A., Al-Dahle A., Letman A., Mathur A., Schelten A., Yang A., Fan A., et al. The llama 3 herd of models // arXiv preprint arXiv:2407.21783. — 2024.
2. Edge D., Trinh H., Cheng N., Bradley J., Chao A., Mody A., Truitt S., and Larson J. From Local to Global: A Graph RAG Approach to Query-Focused Summarization. — 2024. — 2404.16130.
3. Gutiérrez B. J., Shu Y., Gu Y., Yasunaga M., and Su Y. HippoRAG: Neurobiologically Inspired Long-Term Memory for Large Language Models. — 2024. — 2405.14831.
4. Li S., He Y., Guo H., Bu X., Bai G., Liu J., Liu J., Qu X., Li Y., Ouyang W., Su W., and Zheng B. GraphReader: Building Graph-based Agent to Enhance Long-Context Abilities of Large Language Models. — 2024. — 2406.14550.
5. Anokhin P., Semenov N., Sorokin A., Evseev D., Burtsev M., and Burnaev E. AriGraph: Learning Knowledge Graph World Models with Episodic Memory for LLM Agents. — 2024. — 2407.04363.
6. Li B., Fei H., Li F., Wu Y., Zhang J., Wu S., Li J., Liu Y., Liao L., Chua T.-S., and Ji D. DiaASQ : A Benchmark of Conversational Aspect-based Sentiment Quadruple Analysis. — 2023. — 2211.05705.
7. Jin B., Liu G., Han C., Jiang M., Ji H., and Han J. Large Language Models on Graphs: A Comprehensive Survey. — 2024. — 2312.02783.
8. Gu Y. and Su Y. ArcaneQA: Dynamic Program Induction and Contextualized Encoding for Knowledge Base Question Answering. — 2022. — 2204.08109.
9. Li Z., Fan S., Gu Y., Li X., Duan Z., Dong B., Liu N., and Wang J. FlexKBQA: A Flexible LLM-Powered Framework for Few-Shot Knowledge Base Question Answering. — 2024. — 2308.12060.
10. Shu D., Chen T., Jin M., Zhang C., Du M., and Zhang Y. Knowledge Graph Large Language Model (KG-LLM) for Link Prediction. — 2024. — 2403.07311.
11. Baek J., Aji A. F., and Saffari A. Knowledge-Augmented Language Model Prompting for Zero-Shot Knowledge Graph Question Answering. — 2023. — 2306.04136.
12. Luo L., Li Y.-F., Haffari G., and Pan S. Reasoning on Graphs: Faithful and Interpretable Large Language Model Reasoning. — 2024. — 2310.01061.

13. Wen Y., Wang Z., and Sun J. MindMap: Knowledge Graph Prompting Sparks Graph of Thoughts in Large Language Models. — 2024. — 2308.09729.
14. Babaev D., Ovsov N., Kireev I., Ivanova M., Gusev G., Nazarov I., and Tuzhilin A. CoLES: Contrastive Learning for Event Sequences with Self-Supervision // Proceedings of the 2022 International Conference on Management of Data. — ACM. — 2022. — June. — SIGMOD/PODS '22. — P. 1190–1199. — Access mode: <http://dx.doi.org/10.1145/3514221.3526129>.
15. Andoni A., Indyk P., and Razenshteyn I. Approximate Nearest Neighbor Search in High Dimensions. — 2018. — 1806.09823.